LAPORAN PRAKTIKUM

POSTTEST 5

Basis Data

Disusun oleh:

Muhammad Alfauzi Syahputra 2509106006

Kelas A1 ‘25

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

1. Query SQL

a. Create Table

```sql
CREATE TABLE pelanggan (
    id VARCHAR(5) PRIMARY KEY,
    nama VARCHAR(50) NOT NULL,
    alamat VARCHAR(100),
    umur INT NOT NULL,
    saldo DECIMAL(10,2) NOT NULL,
    tanggal_bergabung DATE NOT NULL
);

DESC pelanggan;

CREATE TABLE transaksi (
    id VARCHAR(10) PRIMARY KEY,
    total_pembelian DECIMAL(12,2) NOT NULL,
    waktu_transaksi TIMESTAMP NOT NULL
);

DESC transaksi;
```

Gambar 2.1.1 Create Table

Untuk membuat tabel baru dalam query SQL, digunakan perintah CREATE TABLE. Pada tabel pelanggan, kolom id ditetapkan sebagai primary key, sedangkan kolom alamat bersifat opsional (boleh NULL). Untuk tabel transaksi, digunakan tipe data TIMESTAMP untuk kolom waktu_transaksi. Perintah DESC digunakan untuk melihat struktur kolom dan tipe data dari tabel yang telah dibuat.

	Field	Type	Null	Key	Default	Extra
1	id	varchar(5)	NO	PRI	NULL	EMPTY
2	nama	varchar(50)	NO	EMPTY	NULL	EMPTY
3	alamat	varchar(100)	YES	EMPTY	NULL	EMPTY
4	umur	int	NO	EMPTY	NULL	EMPTY
5	saldo	decimal(10,2)	NO	EMPTY	NULL	EMPTY
6	tanggal_bergabung	date	NO	EMPTY	NULL	EMPTY

Gambar 2.1.2 OUTPUT Create Table pelanggan

	Field	Type	Null	Key	Default	Extra
1	id	varchar(10)	NO	PRI	NULL	EMPTY
2	total_pembelian	decimal(12,2)	NO	EMPTY	NULL	EMPTY
3	waktu_transaksi	timestamp	NO	EMPTY	NULL	EMPTY

Gambar 2.1.3 OUTPUT Create Table transaksi

b. Insert Into

```sql
INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung) VALUES
('RE001', 'Ada Wong', 'Ennerdale Street, Terragrigia', 30, 85000000.00,
'2021-03-17'),
('RE002', 'Leon Scott Kennedy', 'Washington D.C.', 50, 45000.00, '2023-01-30'),
('RE003', 'Grace Ashcroft', 'Old Oak Shopping Center, London', 30, 90000.00,
'2022-06-15'),
('RE004', 'Chris Redfield', 'Boston, Massachusetts', 50, 7250000.00, '2020-07-23'),
('RE005', 'Claire Redfield', 'New York, New York', 50, 5930000.00, '2024-12-24'),
('RE006', 'Albert Wesker', 'Umbrella Research Facility, Europe', 50, 99999999.99,
'2025-05-10'),
('RE007', 'Jill Valentine', 'Arklay Mountains, Raccoon City', 40, 1500000.00,
'2021-08-15'),
('RE008', 'Burton', 'Floating Island, Sheena Island', 50, 2000000.00,
'2023-09-20'),
('RE009', 'Rebecca Chambers', 'Raccoon City Police Department', 40, 800000.00,
'2020-07-24'),
('RE010', 'Alomar', 'West African Village', 40, 1200000.00, '2026-03-05');

SELECT * FROM pelanggan;

INSERT INTO transaksi (id, total_pembelian, waktu_transaksi) VALUES
('TX001', 1500000.00, '2026-04-01 10:30:00'),
('TX002', 75000.00, '2026-04-02 14:22:00'),
('TX003', 2500000.00, '2026-04-03 09:15:00'),
('TX004', 89000.00, '2026-04-04 16:45:00'),
('TX005', 3200000.00, '2026-04-05 11:30:00'),
('TX006', 1200000.00, '2026-04-06 13:05:00'),
('TX007', 500000.00, '2026-04-07 08:50:00'),
('TX008', 4100000.00, '2026-04-08 19:20:00'),
('TX009', 950000.00, '2026-04-09 12:10:00'),
('TX010', 675000.00, '2026-04-10 17:45:00');

SELECT * FROM transaksi;
```

Gambar 2.2.1 insert Into

Untuk memasukkan data ke dalam tabel, digunakan perintah INSERT INTO. Pada tabel pelanggan, digunakan metode multiple query dengan mendefinisikan nama kolom secara eksplisit. Sedangkan pada tabel transaksi, data dimasukkan menggunakan metode single query (satu per satu) tanpa menuliskan nama kolom, sehingga urutan data harus sesuai dengan struktur tabel. Perintah SELECT * digunakan untuk memverifikasi bahwa data telah masuk.

	id	nama	alamat	umur	saldo	tanggal_bergabung
1	RE001	Ada Wong	Ennerdale Street, Terragrigia	30	85000000.00	2021-03-17
2	RE002	Leon Scott Kennedy	Washington D.C.	50	45000.00	2023-01-30
3	RE003	Grace Ashcroft	Old Oak Shopping Center, London	30	90000.00	2022-06-15
4	RE004	Chris Redfield	Boston, Massachusetts	50	7250000.00	2020-07-23
5	RE005	Claire Redfield	New York, New York	50	5930000.00	2024-12-24
6	RE006	Albert Wesker	Umbrella Research Facility, Europe	50	99999999.99	2025-05-10
7	RE007	Jill Valentine	Arklay Mountains, Raccoon City	40	1500000.00	2021-08-15
8	RE008	Burton	Floating Island, Sheena Island	50	2000000.00	2023-09-20
9	RE009	Rebecca Chambers	Raccoon City Police Department	40	800000.00	2020-07-24
10	RE010	Alomar	West African Village	40	1200000.00	2026-03-05

Gambar 2.2.2 OUTPUT insert Into pelanggan

	id	total_pembelian	waktu_transaksi
1	TX001	1500000.00	2026-04-01 10:30:00
2	TX002	75000.00	2026-04-02 14:22:00
3	TX003	2500000.00	2026-04-03 09:15:00
4	TX004	89000.00	2026-04-04 16:45:00
5	TX005	3200000.00	2026-04-05 11:30:00
6	TX006	1200000.00	2026-04-06 13:05:00
7	TX007	500000.00	2026-04-07 08:50:00
8	TX008	4100000.00	2026-04-08 19:20:00
9	TX009	950000.00	2026-04-09 12:10:00
10	TX010	675000.00	2026-04-10 17:45:00

Gambar 2.2.3 OUTPUT insert Into transaksi

c. COUNT

```
SELECT COUNT(*) AS jumlah_pelanggan_gabung_setelah_2022 FROM pelanggan WHERE
YEAR(tanggal_bergabung) > 2022;
SELECT COUNT(*) AS jumlah_transaksi_diatas_1jt FROM transaksi WHERE total_pembelian
> 1000000;
```

Gambar 2.3.1 Sum

Fungsi agregat COUNT(*) digunakan untuk menghitung jumlah baris yang memenuhi kondisi tertentu. Query pertama menghitung jumlah pelanggan (jumlah_pelanggan_gabung_setelah_2022) yang bergabung setelah tahun 2022. Query kedua menghitung jumlah transaksi (jumlah_transaksi_diatas_1jt) yang total pembeliannya melebihi 1.000.000.

	jumlah_pelanggan_gabung_setelah_2022
1	5

Gambar 2.3.2 OUTPUT Count (tanggal_bergabung) > 2022

	jumlah_transaksi_diatas_1jt
1	5

Gambar 2.3.3 OUTPUT Count total_pembelian > 1000000;

d. SUM

```
SELECT SUM(saldo) AS total_saldo_pelanggan_lama FROM pelanggan WHERE
YEAR(tanggal_bergabung) < 2022;
SELECT SUM(total_pembelian) AS total_pembelian_april_2026 FROM transaksi WHERE
YEAR(waktu_transaksi) = 2026 AND MONTH(waktu_transaksi) = 4;
```

Gambar 2.4.1 Sum

Fungsi agregat SUM(kolom) digunakan untuk menjumlahkan nilai-nilai numerik dalam kolom tertentu. Query pertama menjumlahkan total saldo (total_saldo_pelanggan_lama) dari pelanggan yang bergabung sebelum tahun 2022. Query kedua menjumlahkan total pembelian (total_pembelian_april_2026) dari semua transaksi yang terjadi pada bulan April 2026.

	total_saldo_pelanggan_lama
1	94550000.00

Gambar 2.4.2 OUTPUT Sum (tanggal_bergabung) < 2022

	total_pembelian_april_2026
1	14789000.00

Gambar 2.4.3 OUTPUT Sum total_pembelian april 2026

e. AVG

```
SELECT AVG(umur) AS rata_umur_pelanggan_saldo_rendah FROM pelanggan WHERE saldo
< 1000000;
SELECT AVG(total_pembelian) AS rata_pembelian_april_2026 FROM transaksi WHERE
YEAR(waktu_transaksi) = 2026 AND MONTH(waktu_transaksi) = 4;
```

Gambar 2.5.1 AVG

Fungsi agregat AVG(kolom) menghitung nilai rata-rata dari nilai-nilai dalam kolom numerik. Query pertama menghitung rata-rata umur (rata_umur_pelanggan_saldo_rendah) dari pelanggan yang saldonya kurang dari 1.000.000. Query kedua menghitung rata-rata total pembelian (rata_pembelian_april_2026) dari transaksi yang terjadi pada bulan April 2026.

	rata_umur_pelanggan_saldo_rendah
1	40.0000

Gambar 2.5.2 OUTPUT AVG umur saldo < 1000000

	rata_pembelian_april_2026
1	1478900.000000

Gambar 2.5.3 OUTPUT AVG total_pembelian april 2026

f. MIN & MAX

```
SELECT MIN(saldo) AS saldo_terendah, MAX(saldo) AS saldo_tertinggi FROM
pelanggan WHERE tanggal_bergabung > '2021-12-31';
SELECT MIN(total_pembelian) AS min_pembelian_april_2026, MAX(total_pembelian)
AS max_pembelian_april_2026 FROM transaksi WHERE YEAR(waktu_transaksi) = 2026
AND MONTH(waktu_transaksi) = 4;
```

Gambar 2.5.1 MIN & MAX

Fungsi agregat MIN(kolom) dan MAX(kolom) digunakan untuk mencari nilai terkecil dan terbesar dalam kolom tertentu. Query pertama mencari saldo terendah (saldo_terendah) dan saldo tertinggi (saldo_tertinggi) dari pelanggan yang bergabung setelah 31 Desember 2021. Query kedua mencari nilai total pembelian terendah (min_pembelian_april_2026) dan tertinggi (max_pembelian_april_2026) dari transaksi yang terjadi pada bulan April 2026.

	saldo_terendah	saldo_tertinggi
1	45000.00	99999999.99

Gambar 2.5.2 OUTPUT MIN & MAX saldo tanggal_bergabung > '2021-12-31'

	min_pembelian_april_2026	max_pembelian_april_2026
1	75000.00	4100000.00

Gambar 2.5.2 OUTPUT MIN & MAX total_pembelian april 2026

g. GROUP BY & HAVING

```
SELECT YEAR(tanggal_bergabung) AS tahun_gabung, COUNT(*) AS jumlah_pelanggan
FROM pelanggan GROUP BY YEAR(tanggal_bergabung) HAVING COUNT(*) > 1;
SELECT umur, COUNT(*) AS jumlah_pelanggan FROM pelanggan GROUP BY umur HAVING
COUNT(*) > 1;
```

Gambar 2.5.1 GROUP BY & HAVING

GROUP BY digunakan untuk mengelompokkan baris data berdasarkan nilai kolom tertentu. Fungsi agregat seperti COUNT(*) kemudian dihitung untuk setiap grup. Klausa HAVING digunakan untuk menyaring grup-grup yang dihasilkan oleh GROUP BY berdasarkan kondisi yang melibatkan fungsi agregat. Query pertama mengelompokkan pelanggan

berdasarkan tahun bergabungnya dan hanya menampilkan tahun-tahun (tahun_gabung) yang memiliki lebih dari satu pelanggan (jumlah_pelanggan > 1). Query kedua mengelompokkan pelanggan berdasarkan umur dan hanya menampilkan umur (umur) yang dimiliki oleh lebih dari satu pelanggan.

	tahun_gabung	jumlah_pelanggan
1	2021	2
2	2023	2
3	2020	2

Gambar 2.5.2 OUTPUT LIKE BY & HAVING tanggal_bergabung

	umur	jumlah_pelanggan
1	30	2
2	50	5
3	40	3

Gambar 2.5.2 OUTPUT LIKE BY & HAVING umur

h. CASE

```
SELECT id, nama, saldo,
CASE
    WHEN saldo > 50000000 THEN 'High'
    WHEN saldo >= 1000000 THEN 'Medium'
    ELSE 'Low'
END AS kategori_saldo
FROM pelanggan;
```

Gambar 2.5.1 Case

Struktur CASE digunakan untuk mengevaluasi kondisi secara berurutan dan mengembalikan nilai tertentu berdasarkan kondisi yang pertama kali terpenuhi. Query ini mengevaluasi kolom saldo untuk setiap pelanggan dan menetapkan nilai 'High', 'Medium', atau 'Low' ke kolom alias kategori_saldo tergantung rentang nilai saldo masing-masing pelanggan. Jika saldo lebih dari 50 juta, maka kategorinya adalah 'High'. Jika tidak, dan saldo lebih besar atau sama dengan 1 juta, kategorinya adalah 'Medium'. Jika tidak memenuhi keduanya, maka kategorinya adalah 'Low'.

	id	nama	saldo	kategori_saldo
1	RE001	Ada Wong	85000000.00	High
2	RE002	Leon Scott Kennedy	45000.00	Low
3	RE003	Grace Ashcroft	90000.00	Low
4	RE004	Chris Redfield	7250000.00	Medium
5	RE005	Claire Redfield	5930000.00	Medium
6	RE006	Albert Wesker	99999999.99	High
7	RE007	Jill Valentine	1500000.00	Medium
8	RE008	Burton	2000000.00	Medium
9	RE009	Rebecca Chambers	800000.00	Low
10	RE010	Alomar	1200000.00	Medium

Gambar 2.5.2 OUTPUT LIKE Case